

# Health checks

`calepin health` runs quick diagnostic checks on the current project (notebook or website) and reports anything that could break rendering or execution.

It checks the following by default:

- executable availability for configured engines and diagram tools
- Python/Jupyter kernel metadata consistency
- local literal links inside `.typ` files
- local literal images inside `.typ` files
- missing alt text on literal Typst images
- duplicate explicit page routes from `slug` and `url` page metadata

```
calepin health [OPTIONS]
```

Use `--json` for machine-readable output.

## Link checking

`health scans #link("...")` literals in Typst source files and verifies targets.

- A bad local target (for example `#link("missing.html")`) is reported as an error.
- External `http://` and `https://` links are ignored by default.
- Pass `--check-external-links` to validate HTTP(S) links too.

By default, `health` walks directories recursively from the current directory, ignoring `.calepin`, `.git`, `target`, `node_modules`, and `.venv`. You can limit recursion depth with `--depth`:

```
calepin health --depth 2
```

With recursive links that may refer to generated outputs, run `calepin health` after a build so referenced pages exist.

## Image checking

`health` scans literal `#image("...")` calls in Typst source files. Missing local image files are reported as errors. Images without non-empty `alt:` text are reported as warnings.

Relative image paths are resolved from the source file that contains the image call. Root-relative paths are resolved from the project root. External and special targets such as `http://`, `https://`, `data:`, `mailto:`, and fragment-only targets are ignored.

Alt text is considered present when the image call has an `alt:` argument with a non-empty string or a computed value. These calls pass the accessibility check:

```
#image("assets/chart.png", alt: "Line chart of quarterly revenue")
#image("assets/logo.svg", alt: figure_alt)
```

These calls produce warnings:

```
#image("assets/chart.png")
#image("assets/chart.png", alt: "")
#image("assets/chart.png", alt: none)
```

## Page route checking

health scans `<website-metadata>` for literal `slug` and `url` values and reports duplicate or unsafe output routes as errors. The check compares the rendered HTML route, so pages in different directories can use the same slug as long as they do not write the same output path.

For example, these two pages collide because they both render to `same.html`:

```
#metadata((slug: "same")) <website-metadata>
#metadata((url: "same.html")) <website-metadata>
```

Routes that escape the output directory are also errors:

```
#metadata((slug: "../outside")) <website-metadata>
#metadata((url: "/absolute/path")) <website-metadata>
```

## Source scanning

Raw Typst code spans and blocks are ignored by source checks, so documentation examples do not need to point at real files.

The source checks are intentionally conservative: they only inspect literal `#link("...")`, `#image("...")`, `slug: "..."`, and `url: "..."` values. Dynamic expressions are left to the normal build process.

`--strict` turns warnings into failures (in addition to errors).

```
calepin health --strict --check-external-links
```